turbogrid

Building synthetic spectral grids with Turbospectrum + MARCS,
and turning them into FERRE grids

User guide — general workflow, with a worked APOGEE example

What this guide covers. How to go from a set of stellar-parameter axes (T_{eff}, log g, [M/H], [C/M], [α/M], [N/M], ...) to a complete, continuum-normalised, FERRE-ready synthetic spectral grid, running the synthesis in parallel on a workstation or a Slurm cluster. Sections 1–9 are generic and apply to any wavelength range or instrument. Section 10 is a worked example for APOGEE (H band, R ~ 22,500).

Files involved.

File	Role
`turbogrid.py`	The library: build grid table, run points, collect, convolve, write FERRE grid.
`fill_grid_holes.py`	Standalone (numpy/scipy only): RBF-interpolates any missing grid nodes so the grid is complete.
`synthesis.py`	Your Turbospectrum wrapper exposing `synthesize()`. Imported by turbogrid.
`atmos.py`, `utils.py`	Holtzman's atmosphere utilities (filename(), marcs2turbo(), kurucz2turbo(), fill_holes()).
`build_turbogrid.py`	Your driver script for steps 1–2 (grid table + job script).
`collect_turbogrid.py`, `write_ferre_grid.py`	Your driver scripts for steps 4–7.

1. The pipeline at a glance

Step	Function	Where it runs	Output
1. Build grid table	make_atmod_lookup(), make_grid_params(), write_grid_table()	Login node / interactive	grid_params.txt, grid_meta.pkl
2. Write job script	write_slurm_array() or write_parallel_joblist()	Login node	submit_array.slurm / joblist.txt
3. Synthesise	run_one() per point (via Slurm / GNU parallel / build_grid_local())	Compute nodes	outputs_dir/NNNNN/spec_NNNNNN.npz
4. Collect	collect_grid()	One node, plenty of RAM	grid.npz
5. Fill holes	fill_grid_holes.py	One big-memory node (or a small array)	grid_filled.npz
6. Convolve (optional)	convolve_grid_to_R()	One node	grid_R.npz
7. FERRE grid	write_ferre_grid()	One node	ASCII FERRE synth file
8. Validate	ascii2bin + test fits	Anywhere	Confidence

The key design idea: every grid point is a **row in one table**, and a single generic function `run_one(outdir, idx)` computes row *idx*. Any parallel scheme just has to hand out integers. A point whose output file already exists is skipped, so **restarting is always safe**: resubmit the same job and only unfinished points are done.

2. Prerequisites

- Python 3 with **numpy** and **scipy**.
- A working **Turbospectrum** installation (babsma + bsyn) that your `synthesis.synthesize()` can call. Test a single spectrum by hand first.
- An atmosphere grid (MARCS or Kurucz) on disk, readable by `atmos.filename()`.
- Line lists (atomic, molecular, hydrogen) covering your wavelength range, in Turbospectrum format.
- `turbogrid.py`, `fill_grid_holes.py`, `synthesis.py`, `atmos.py`, `utils.py` in the same directory, or on `PYTHONPATH`. If your synthesis module has a different name, edit the `import synthesis` line at the top of turbogrid.py.
- For clusters: Slurm (or GNU parallel), and a **data/scratch filesystem** with room for one file per grid point (see Section 4).

> **Before anything large:** run a tiny grid (e.g. 2 x 2 x 2 points) end-to-end with `build_grid_local()` and inspect a spectrum by eye. It takes minutes and catches path, linelist and atmosphere problems before you burn thousands of CPU-hours.

3. Step 1 — define the grid

3.1 Atmosphere lookup

```
import turbogrid as tg

lookup = tg.make_atmod_lookup(
    griddir='/path/to/marcs_models',   # root of the raw atmosphere grid
    model='marcs',                     # or 'kurucz'
    flat=True)                         # True if all model files sit directly in griddir
```

The lookup converts each needed model to Turbospectrum format once and caches it in `griddir/_turbo_cache` (override with `cache_dir=`). It falls back to `*.filled` models made by

`atmos.fill_holes()`. Combinations with no model at all are **dropped** with a printed warning — they are filled later at the spectrum stage (Section 7).

3.2 Grid axes

Each axis is either a list of values or a FERRE-style triplet `(n, start, step)`.

```
params = tg.make_grid_params(
    teff_grid=(n_teff, teff0, dteff),   # e.g. (15, 3000., 250.) -> 3000..6500 K
    logg_grid=(n_logg, logg0, dlogg),
    mh_grid=(n_mh, mh0, dmh),
    cm=(n_cm, cm0, dcm),                # [C/M]   -- changes the ATMOSPHERE choice
    am=(n_am, am0, dam),                # [a/M]   -- changes the ATMOSPHERE choice
    nm=(n_nm, nm0, dnm),                # [N/M]   -- synthesis only
    vmicro='holtzman',                  # or a fixed value, or another axis
    atmod_lookup=lookup)
print(len(params), 'grid points')
```

- Pass a **scalar** for `am`/`cm`/`nm`/`vmicro` to hold it fixed; pass a list/triplet to make it an axis.
- `vmicro='holtzman'` sets v_{micro} = 2.478 – 0.325 log g. It is then *tied to log g*, not a free axis.
- Grid size = product of all axis lengths (minus any atmosphere-less combinations). It grows fast: check the printed count before going further.

3.3 Write the grid table

```
tg.write_grid_table(
    params, outdir='mygrid',             # small control files live here
    wrange=(w_start, w_end), dw=0.01,    # synthesis range & step, Angstrom
    linelists=[atomic_list, molecular_list, hydrogen_list],
    atmos_type='marcs',
    outputs_dir='/data/me/mygrid_outputs',   # BIG: one file per point
    shard_size=1000,                         # files per subdirectory
    # optional, convolve each spectrum on the fly (Section 8):
    # R_out=22500, dw_out=0.1, wout=(w1, w2),
    # optional abundance overrides passed to synthesize(elems=...):
    # elems=...
)
```

Pad wrange. If you will convolve to lower resolution later, make `wrange` a few resolution elements wider than the range you actually want, so convolution edge effects fall outside it. With on-the-fly convolution (`R_out`), `wout` must lie inside `wrange`.

4. Step 2 — storage and job layout

4.1 Where outputs go

- `outdir` holds only `grid_params.txt`, `grid_meta.pkl`, the Slurm script and logs. Home/project space is fine.
- `outputs_dir` holds one `.npz` per point. Put it on data or scratch storage. Its absolute path is stored in grid_meta.pkl, so jobs find it regardless of their working directory.
- Outputs are **sharded** into subdirectories of `shard_size` files (`00000/`, `00001/`, ...). Never put ~10^6 files in one directory on a shared filesystem.
- Rough disk budget: n_points x n_pix x 3 arrays x 8 bytes. A 10^6-point grid at dw = 0.01 Å over 2000 Å (200k pix) is ~5 TB unconvolved — another reason to consider on-the-fly convolution (Section 8).

4.2 Slurm array job (recommended on clusters)

```
tg.write_slurm_array('mygrid', jobname='mygrid',
                     time='24:00:00', mem='8G', partition='nodes',
                     array_step=K,          # grid points per array task
```

```
                    max_concurrent=30,     # tasks running at once (etiquette)
                    max_array_size=999)    # your cluster's MaxArraySize
```

Choosing `array_step`: the number of array tasks is ceil(N / array_step), and it must not exceed the cluster's MaxArraySize (`scontrol show config | grep MaxArraySize`). The minimum is **array_step = ceil(N / MaxArraySize)**; if it is too small the function raises and tells you the minimum.

Choosing `time`: each task runs array_step points *serially*. Time one point (t_1, from a test run) and request comfortably more than array_step x t_1. If tasks hit the wall clock, just resubmit — finished points are skipped. Alternatively use `parallel=P` (with `array_step >= P`) to run P points at once inside each task on P cores.

```
sbatch mygrid/submit_array.slurm
squeue -u $USER
```

4.3 Alternatives

```
# One multi-core machine:
tg.build_grid_local('mygrid', nproc=32)

# GNU parallel:
tg.write_parallel_joblist('mygrid')
#   then on the shell:  parallel -j 32 :::: mygrid/joblist.txt
```

5. Step 3 — monitoring and restarting

```
# how many finished / failed
find /data/me/mygrid_outputs -name 'spec_*.npz'    | wc -l
find /data/me/mygrid_outputs -name 'spec_*.failed' | wc -l

# see where a task is actually writing (run_one prints the resolved path)
grep outfile mygrid/slurm_<jobid>_<task>.out | tail

# read why a point failed
cat /data/me/mygrid_outputs/00012/spec_012345.failed
```

- **Restart:** resubmit the same `submit_array.slurm`. Existing outputs are skipped; `.failed` files are retried and removed on success.
- **Do not edit grid_params.txt mid-run.** Row numbers are the identity of each point. If you change the grid, write a new table into a new `outdir` and `outputs_dir`.
- If a whole block of points fails the same way, it is almost always a missing atmosphere, a bad linelist path, or Turbospectrum running out of memory/temp space — fix the cause, then resubmit.

6. Step 4 — collect into one file

```
tg.collect_grid('mygrid', gridfile='/data/me/mygrid_outputs/grid.npz')
```

Reads every finished spectrum, checks they share one wavelength grid, and writes one `.npz` with `params` (structured array), `wave`, `flux`, `cont`. It reports how many points are missing or failed. Note that `gridfile` is joined onto `outdir`, so an absolute path is used as given. Run it on a node with enough RAM to hold the whole grid (n_points x n_pix x 2 x 8 bytes).

7. Step 5 — fill holes

FERRE needs **every** combination of axis values present. Points can be missing because no atmosphere existed, synthesis failed, or a job was still running. `fill_grid_holes.py` fills them by RBF interpolation from neighbouring spectra in (index-scaled) parameter space, flags them (`interpolated`, and `edge_fill` for extrapolated edge points), and never re-synthesises.

```
# single big-memory node
python fill_grid_holes.py grid.npz \
```

```
    --labels teff logg mh cm am nm \
    --neighbors 100 --degree 0 --nproc 4 --dtype float32 \
    --outfile grid_filled.npz
```

Memory is the constraint, not cores. Each worker holds its own float64 copy of flux and cont: ~2 x n_real_points x n_pix x 8 bytes *per worker* (e.g. ~31 GB at 857k points x 2401 pix). Set `--nproc` from your `--mem`, and always pass it explicitly on Slurm. If one node is not enough, split across an array and merge:

```
# array task i of N
python fill_grid_holes.py grid.npz --labels teff logg mh cm am nm \
    --nproc 2 --dtype float32 --n-chunks N --chunk-index $SLURM_ARRAY_TASK_ID \
    --outfile chunk${SLURM_ARRAY_TASK_ID}.npz
# afterwards
python fill_grid_holes.py grid.npz --labels teff logg mh cm am nm \
    --merge chunk0.npz chunk1.npz ... --outfile grid_filled.npz
```

- `--labels` must list exactly the axes that define the regular grid. **Do not include vmicro** if you used `vmicro='holtzman'` — it varies with log g and is not an independent axis.
- Check the reported fraction of interpolated points. A few per cent of interior points is routine; large contiguous holes, or many edge fills, mean you should fix the atmosphere coverage instead.
- Downstream, consider masking or down-weighting fits that land near `edge_fill` nodes.

8. Step 6 — convolve to instrument resolution

Two options; use one, not both.

- **Post hoc** (on the collected grid): simple, but you must store and collect the full-resolution grid first.
- **On the fly**: set `R_out`, `dw_out`, `wout` in `write_grid_table()`. Each point is convolved right after synthesis and saved only at output resolution — far less disk and RAM. Recommended for big grids.

```
tg.convolve_grid_to_R('grid_filled.npz', 'grid_R.npz', R=R_instrument, dw_out=dw_out)
```

Convolution uses a Gaussian of constant FWHM in ln λ (constant R). Pick `dw_out` to give at least ~2–3 pixels per resolution element at the blue end (λ_{min} / R / dw_out >= 2). If your instrument has a non-Gaussian or varying LSF, this step is an approximation — convolve with the real LSF instead if it matters for your science.

9. Step 7 — write and validate the FERRE grid

```
from turbogrid import write_ferre_grid
write_ferre_grid('grid_R.npz', 'mygrid.ferre',
                 labels=('teff', 'logg', 'mh', 'cm', 'am', 'nm'),
                 normalize=True,          # divide by continuum
                 resolution=R_instrument,
                 vacuum=0,                # 1 if your wavelengths are vacuum
                 comment='short description of this grid')
```

The writer checks that every axis is evenly spaced, that the grid is complete, and that the wavelength grid is linear (it writes LOGW = 0). Axes that do not vary are dropped automatically.

Check these three things before trusting the output.
1. **Axis order.** The docstring says `labels[0]` is fastest-varying, but the stride code makes the *last* label fastest-varying. Make sure the order matches what FERRE expects (compare against an existing grid of yours), and keep it consistent with the order used in fill_grid_holes.
2. **vmicro.** The default `labels` includes `vmicro`. With Holtzman vmicro it has many unique values and the completeness check will fail — pass `labels` without it.
3. **VACUUM flag.** Set it to match your line lists' wavelength convention, not Turbospectrum's default.

Validation recipe:

- `ascii2bin mygrid.ferre` (from the FERRE distribution) — it should load without complaint.
- Run FERRE in pure-interpolation mode at a few **real** grid nodes and compare against the stored spectra.
- Fit a handful of grid spectra (with noise added) and check the recovered labels.
- Fit a few well-studied real stars and compare to literature/survey values.

10. Worked example: an APOGEE-like grid

APOGEE observes 1.51–1.70 μm at R ~ 22,500, in **vacuum** wavelengths, and ASPCAP grids are parameterised in T_{eff}, log g, [M/H], [α/M], [C/M], [N/M] (plus v_{micro} and broadening). The values below are illustrative — match your axis ranges to your science case and your atmosphere coverage.

10.1 Build the table and job (build_grid_apogee.py)

```
import turbogrid as tg

linelists = ['/path/to/molecular_vac.list',      # APOGEE-format, vacuum
             '/path/to/atomic_vac.list',
             '/path/to/Hlinedata']

lookup = tg.make_atmod_lookup(griddir='/path/to/marcs_apogee',
                              model='marcs', flat=True)

params = tg.make_grid_params(
    teff_grid=(15, 3000., 250.),     # 3000-6500 K
    logg_grid=(12, -0.5, 0.5),       # -0.5..5.0
    mh_grid=(14, -2.5, 0.25),        # -2.5..+0.75
    cm=(11, -1.5, 0.25),             # -1.5..+1.0
    am=(7, -1.0, 0.25),              # -1.0..+0.5
    nm=(6, -0.5, 0.5),               # -0.5..+2.0
    vmicro='holtzman',
    atmod_lookup=lookup)
N = len(params); print(N, 'grid points')

tg.write_grid_table(
    params, outdir='grid_apogee',
    wrange=(15050., 17050.), dw=0.01,          # padded around 15100-17000
    linelists=linelists, atmos_type='marcs',
    outputs_dir='/data/me/grid_apogee_out',
    shard_size=1000,
    R_out=22500, dw_out=0.1, wout=(15100., 17000.))   # convolve on the fly

tg.write_slurm_array('grid_apogee', jobname='apogrid',
                     time='48:00:00', mem='8G', partition='nodes',
                     array_step=-(-N // 999),   # = ceil(N/999)
                     max_concurrent=30, max_array_size=999)
```

With these axes the full product is 1,164,240 combinations (fewer after atmosphere-less ones are dropped). With MaxArraySize = 999, array_step is ~1,000–1,170 points per task, run serially — so set `time` from a measured per-point runtime, or use `parallel=` to run several points per task concurrently.

> 15050–17050 Å padding: at R = 22,500 one FWHM is ~0.7 Å, so 50 Å of padding is ample. dw_out = 0.1 Å gives ~7 pixels per FWHM at the blue end. Output: 19,001 pixels per spectrum.

10.2 Run, collect, fill, write

```
sbatch grid_apogee/submit_array.slurm
# ...wait; resubmit the same script if any tasks time out...

import turbogrid as tg
tg.collect_grid('grid_apogee', gridfile='/data/me/grid_apogee_out/grid.npz')

python fill_grid_holes.py /data/me/grid_apogee_out/grid.npz \
    --labels teff logg mh cm am nm --dtype float32 --nproc 2 \
    --outfile /data/me/grid_apogee_out/grid_filled.npz

from turbogrid import write_ferre_grid
write_ferre_grid('/data/me/grid_apogee_out/grid_filled.npz',
                 '/data/me/grid_apogee_out/apogee_marcs.ferre',
                 labels=('teff', 'logg', 'mh', 'cm', 'am', 'nm'),  # no vmicro
                 normalize=True, resolution=22500,
                 vacuum=1,                      # vacuum line lists
                 comment='APOGEE MARCS grid, R=22500')
```

Memory check for the APOGEE case. ~10^6 points x 19,001 pixels x 2 arrays x 8 bytes ~ 300 GB per fill_grid_holes worker. Either use the chunked mode across several big-memory nodes, build per-sub-grid fills (e.g. per [M/H] slab), or reduce the pixel count (coarser dw_out, or fill on a trimmed wavelength range). Also note that ASPCAP fits APOGEE spectra on a log-linear pixel grid; this writer produces a linear-λ grid (LOGW = 0), so resample your observed spectra (or the grid) consistently.

10.3 Changes from a generic run, in one table

Setting	Generic	APOGEE
wrange / wout	Your range, padded	15050–17050 / 15100–17000 Å
Line lists	Any Turbospectrum format	APOGEE vacuum lists + H lines
R_out, dw_out	Instrument R; >= 2–3 pix/FWHM	22,500; 0.1 Å
vacuum	Match line lists	1
vmicro	Fixed, axis, or holtzman	'holtzman' (exclude from labels)
Axes	Science-driven	Teff, logg, [M/H], [C/M], [a/M], [N/M]

11. Troubleshooting

Symptom	Likely cause / fix
Many "no atmosphere for teff=..." warnings	Grid extends beyond model coverage, or wrong `flat/griddir`. Check one path by hand; run atmos.fill_holes() for gaps; or trim the axis.
write_slurm_array raises "exceeds max_array_size"	Increase array_step to the minimum the error message prints.
Tasks hit time limit	Resubmit (finished points skip), lengthen time, or use parallel=P.
Outputs not appearing where expected	grep the job log for "outfile=": run_one prints the resolved path. outputs_dir comes from grid_meta.pkl, not the working directory.
collect_grid: "different wavelength grid"	Mixed runs with different wrange/dw/R_out. Use one consistent outputs_dir per grid.
fill_grid_holes killed (OOM)	Lower --nproc, use --dtype float32, use chunked mode, or fewer pixels.
"Singular matrix" in fill_grid_holes	Keep --degree 0 (default).
write_ferre_grid: "grid is not complete/regular"	Run fill_grid_holes first; remove vmicro (or any dependent quantity) from labels; check rounding of axis values.
write_ferre_grid: "axis not evenly spaced"	An axis has a missing value entirely (a whole slab failed) or was defined unevenly.
FERRE fits look systematically wrong	Check axis order (Section 9), VACUUM flag, normalisation, and wavelength-grid consistency with the data.